241080076 Vishwajeet Waghmare | 241080077 Awwab Wadekar
241080078 Yash Wadhwa | 241080082 Abiraj Kangotra

Hybrid Offline-First HMS: Project Estimation

6.Requirement Analysis Modeling.

1. Introduction & System Context

System Scope: A Hybrid Offline-First Hospital Management System (HMS) designed to maintain uninterrupted clinical operations during network outages. It features local SQLite database caching on Windows desktop clients, real-time queue broadcasts to embedded Raspberry Pi/ESP32 display nodes over local LAN, and background synchronization to a cloud FastAPI server.

2. Data Modeling (Entity-Relationship & Schema Structure)

The data model structures primary healthcare entities, queue management logic, and offline sync tracking:

Entity Name	Attributes (PK / FK / Fields)	Entity Relationships	Offline & Sync Constraints
Patient	patient_id (PK, UUID), full_name, dob, gender, contact_no, created_at	1-to-Many with **Appointment** & **Queue Token**	UUID v4 generated locally to prevent primary key collisions during offline creation.
Queue Token	token_id (PK, UUID), patient_id (FK), dept_id, token_num, status, timestamp	Many-to-1 with **Patient** & **Department**	Direct local broadcast via WebSocket/MQTT to embedded display nodes.
Sync Audit Log	sync_id (PK), entity_name, record_id, action_type, sync_status, updated_at	1-to-1 with local DB mutation transactions	Tracks pending local mutations (PENDING, SYNCED, CONFLICT) for cloud push.

3. Functional Modeling (Data Flow Diagrams - DFD)

A. Level 0 Context Diagram

- **External Entities:** Receptionist, Doctor, Embedded Patient Display Node, Cloud API Server.
- **Input Data Streams:** Patient registration details, queue status updates, clinical triage vitals.
- **Output Data Streams:** Printed token slips, live LAN queue display socket updates, HTTPS sync payloads.

B. Level 1 Process Breakdown

- **Process 1.0 (Patient Management):** Validates and stores patient demographic records in local encrypted SQLite.
- **Process 2.0 (Queue & Display Engine):** Generates sequential queue tokens and broadcasts display state changes over local LAN.
- **Process 3.0 (Sync Handler Engine):** Logs local database mutations and executes background push/pull cycles with cloud endpoints upon network reconnection.

4. Behavioural Model (State Transitions & System Dynamics)

The behavioral model captures state changes across varying network conditions and operational events:

Subsystem State Machine	Initial State	Trigger Event / Input	Next State & System Action
Sync Engine Lifecycle	OFFLINE_LOCAL	Network ping success / Connection restored	**State: SYNCING** Flushes pending audit logs to FastAPI cloud API via HTTPS TLS.
Queue Token Lifecycle	ISSUED	Doctor triggers "Call Next" in desktop UI	**State: CALLED / IN_CONSULT**

			Broadcasts UDP/Socket payload to update Pi/ESP display screen.
Conflict Handling	SYNCING	Cloud API reports version conflict	**State: RESOLVING_CONFLICT** Executes *Last-Write-Wins* timestamp policy and reconciles DB.

5. Prepare Data Dictionary

Comprehensive data element definitions for local SQLite tables and API JSON payloads:

Data Name	Data Type	Field Format / Range	Description & Usage Definition
patient_id	String (UUIDv4)	36 chars, alphanumeric	Unique client-generated identifier; guarantees global uniqueness offline.
token_status	Enum	WAITING | CALLED | DONE	Queue status indicator; controls display color and sound on Raspberry Pi.
sync_flag	Integer (Enum)	0 (Pending), 1 (Synced)	Bit indicator in local SQLite database marking records awaiting cloud sync.

last_updated	Timestamp	ISO 8601 UTC string	Microsecond-precision timestamp used for conflict resolution (*Last-Write-Wins*).

6. Software Requirement Specification (SRS)

6.1 Functional Requirements (FR)

- **FR-01 (Offline Autonomous Operation):** The system shall enable full patient registration and queue token assignment locally without active internet connectivity.
- **FR-02 (Low-Latency Display Broadcasting):** The desktop application shall update live queue screens on Raspberry Pi/ESP32 display nodes over local LAN in **<500ms**.
- **FR-03 (Background Sync Execution):** The background sync service shall auto-detect network restoration and synchronize pending local transactions to the cloud without blocking desktop UI interactions.
- **FR-04 (Deterministic Conflict Resolution):** The cloud sync engine shall resolve concurrent record modifications using microsecond timestamp-based *Last-Write-Wins* logic.

6.2 Non-Functional Requirements (NFR)

- **NFR-01 (Performance & Latency):** Local SQLite query execution times shall remain **<10ms** for 99% of transactions; desktop application launch time shall be under 3 seconds.
- **NFR-02 (Data Security & Privacy):** Local SQLite database files shall be encrypted at rest using SQLCipher; cloud API transport must strictly enforce HTTPS/TLS 1.3.
- **NFR-03 (Reliability & Availability):** The desktop application shall provide **100% operational uptime** for reception check-in workflows during WAN internet outages.
- **NFR-04 (Portability & Build Target):** The core desktop client shall be compiled into a standalone Windows .exe installer package for simple deployment.

7. Conclusion

This Requirement Analysis Modeling and Software Requirement Specification (SRS) document provides a structured structural and behavioral blueprint for the Hybrid Offline-First Hospital Management System[cite: 4]. By integrating robust Data Modeling (UUID-driven SQLite schemas), Functional Data Flow Diagrams (DFDs), Behavioural State Machines, and a precise Data Dictionary, the specification ensures seamless local operational autonomy and deterministic cloud synchronization[cite: 4, 5, 6]. Enforcing these functional and non-functional requirements guarantees high system reliability, low-latency display updates, and absolute data integrity across both offline and online clinical environments[cite: 5, 6].